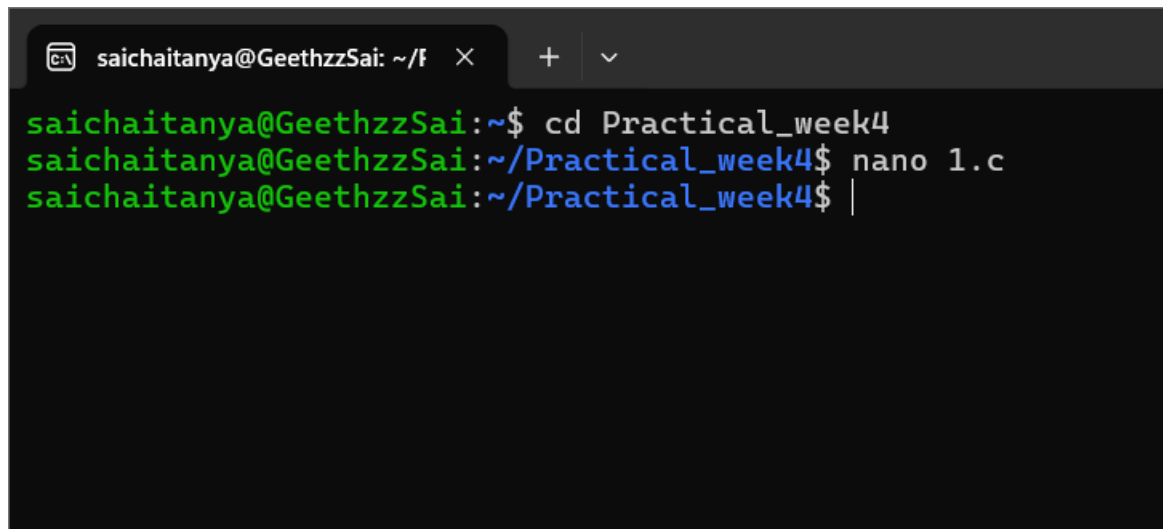


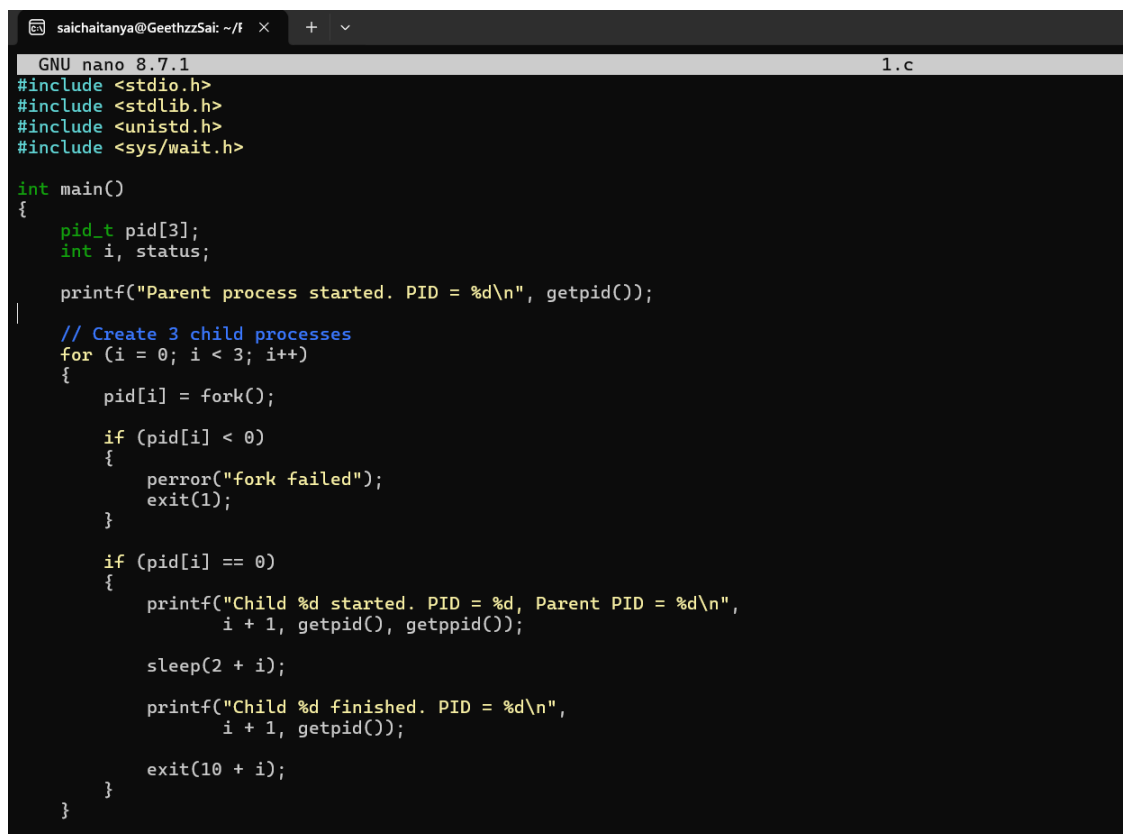
WEEK 4

TASK 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions .

1) Create a file

A terminal window with a dark background. The prompt is 'saichaitanya@GeethzzSai: ~/f'. The user enters 'cd Practical_week4', then 'nano 1.c', and finally a vertical bar '|' to start editing the file.

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week4
saichaitanya@GeethzzSai:~/Practical_week4$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week4$ |
```

A terminal window showing the GNU nano 8.7.1 editor editing the file '1.c'. The code creates three child processes using fork(), sleeps for different durations, and then uses wait() to wait for all children to finish before exiting.

```
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid[3];
    int i, status;

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());

            sleep(2 + i);

            printf("Child %d finished. PID = %d\n",
                   i + 1, getpid());

            exit(10 + i);
        }
    }
}
```

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week4
saichaitanya@GeethzzSai:~/Practical_week4$ nano 1.c
saichaitanya@GeethzzSai:~/Practical_week4$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/Practical_week4$ ./1
Parent process started. PID = 619
Child 1 started. PID = 620, Parent PID = 619

Child 2 started. PID = 621, Parent PID = 619
Parent is waiting using wait()...
Child 3 started. PID = 622, Parent PID = 619
Child 1 finished. PID = 620
wait() collected child PID = 620
Child exit status = 10

Parent is waiting using waitpid()...
Child 2 finished. PID = 621
waitpid() collected child PID = 621
Child exit status = 11
Child 3 finished. PID = 622
waitpid() collected child PID = 622
Child exit status = 12

All child processes completed.
saichaitanya@GeethzzSai:~/Practical_week4$ |
```

TASK 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week4
saichaitanya@GeethzzSai:~/Practical_week4$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week4$ |
```

```

GNU nano 8.7.1                2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process running. PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent process running. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping for 30 seconds...\n");
        sleep(30);

        printf("Parent exiting...\n");
    }

    return 0;
}

```

```

saichaitanya@GeethzzSai: ~/f × + v
saichaitanya@GeethzzSai:~$ cd Practical_week4
saichaitanya@GeethzzSai:~/Practical_week4$ nano 2.c
saichaitanya@GeethzzSai:~/Practical_week4$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/Practical_week4$ ./2
Parent process running. PID = 870
Child PID = 871
Parent sleeping for 30 seconds...
Child process running. PID = 871
Child process exiting...
|

```